

Lab 6 - DLX Memory & Writeback

Nate Herbst
A02307138
Nathan Walker
A02364124

Introduction

This project required implementing the memory and write back stages, the last 2 stages of the DLX processor pipeline. These stages contain RAM and some simple control logic implemented with muxes. The memory stage will receive the ALU result and determine if it needs to store it. If not, pass it to the write back stage. Write back stage will receive some values and with some control logic to determine what data to write to the specified register. At this stage we now have a full functioning DLX processor that can compute factorials.

Procedure

Modular Systems:

The last two stages of the processor were relatively simple with little additional implementation needed beyond some behavioral logic. We implemented the IP RAM block in memory but that was the biggest thing instantiated. We also used muxes and registers from previous labs.

Challenges and Solutions:

1. Branches were not taken as well as registers not being written to. As we started to go through the processor pipeline we ran into unexpected behavior. Data was not being written back to registers correctly or branches were not being taken. This led to our program running into a loop at the beginning and not computing the rest of the factorial. However, we found there were some mismatches in how we were parsing the instruction and from how we wrote the assembler. After fixing these errors we were able to get the processor working.
2. The second challenge we ran into was finding out that we had made and been using an incorrect version of our factorial code. This version incorrectly computed the factorial. This caused some headache for us originally thinking that it was our DLX Processor that had a bug, we didn't stop to consider that we just wrote a bad factorial program. After fixing it and regenerating the MIF file, we were able to get it working.

Results

The complete five-stage DLX pipeline processor was verified in Questa by running a NOP-padded factorial program with an input value of 5 stored at data memory address 0x001. The simulation waveform confirmed correct operation across all pipeline stages.

In the fetch stage, the program counter was observed incrementing sequentially through NOP-padded instructions and jumping to the correct target addresses when branches and jumps were taken—for example, JAL at address 0x021 redirected the PC to the factorial subroutine at 0x034, and JR at 0x053 returned to the saved link address in R31.

In the decode stage, the register file read the correct source operand values on each clock edge, and the sign extender properly produced 32-bit immediates for I-type instructions such as ADDI and SUBI.

The execute stage ALU computed correct arithmetic results, and the branch checker asserted branch enable for BEQZ (when R7 reached zero), BNEZ (while the multiply loop counter R8 was nonzero), and all unconditional jumps (J, JAL, JR).

The memory stage successfully performed load and store operations to the data RAM: the LW instruction at address 0x004 read the input value 5 from RAM address 0x001 into R5, and the SW instruction at address 0x04A wrote the accumulated factorial result back to RAM address 0x000.

The write-back stage correctly selected between the RAM output (for LW) and the ALU result (for arithmetic instructions) via its MUX, and asserted write enable only for register-writing instructions while disabling it for NOPs, stores, and branches.

The write-back stage also correctly handled JAL by writing the return address (PC+1) to R31. After the full program executed—looping through the outer loop 4 times and the inner multiply loop for each iteration—the final value written to data memory address 0x000 was 0x78 (120 in decimal), confirming the correct computation of 5 factorial.

The 4-NOP padding before and after every load, store, branch, and jump instruction successfully prevented all data and control hazards without requiring forwarding or stalling logic.

Figures

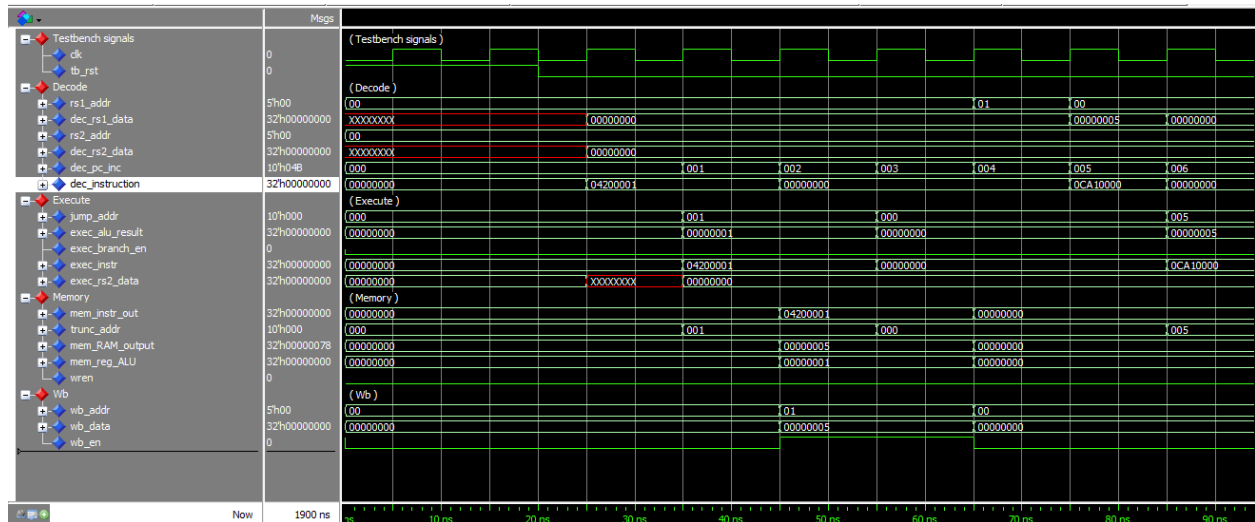


Figure 1: Middle of the factorial operation with our extensive Questa Sim

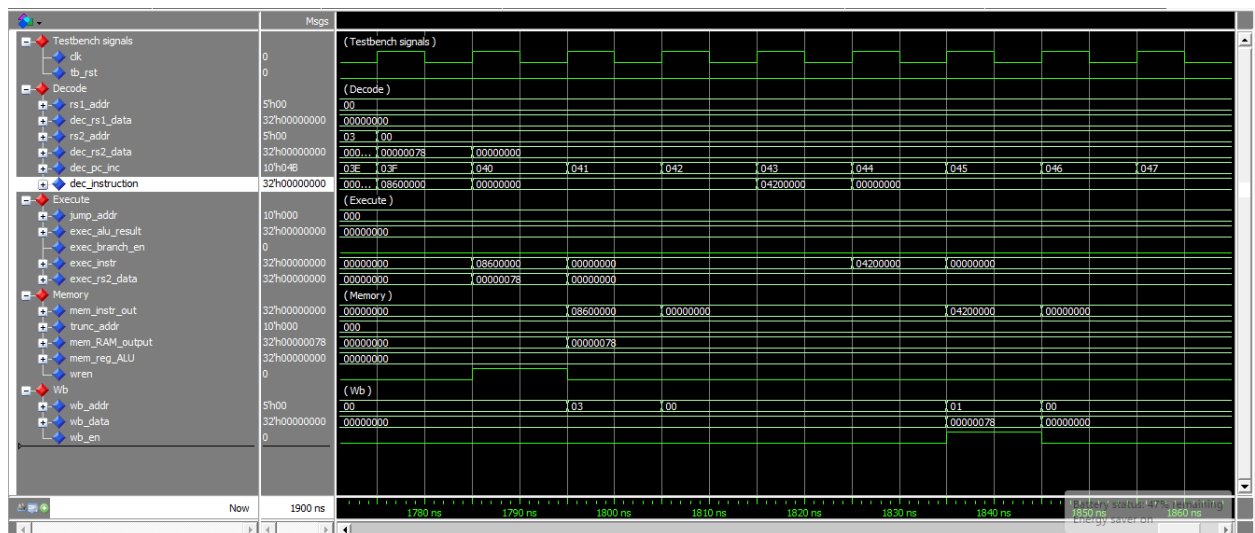


Figure 2: tb_branch_check.vhd Testbench simulation

Conclusion

This lab brought the processor to the point where it can actually perform computation rather than just preparing to do so. Implementing the execute stage reinforced the importance of keeping pipeline register boundaries consistent across all signals, and highlighted the impact of data and control hazards on correctness. The NOP-insertion workaround demonstrated why forwarding and stalling are essential for real pipeline performance. These lessons will directly apply as we add the memory and writeback stages in future labs.

Appendix

DLX_Processor.vhd (top level file)

```
library ieee;
use ieee.std_logic_1164.all;

library work;
use work.fetch_pkg.all;

entity DLX_Processor is
    generic (
        WIDTH          : integer := 10;
        INSTR_WIDTH     : integer := 32
    );
    port (
        clk             : in  std_logic;
        rst             : in  std_logic
    );
end entity DLX_Processor;

architecture structural of DLX_Processor is
    -- Fetch -> Decode signals
    signal internal_pc_inc : std_logic_vector(WIDTH-1 downto 0);
    signal internal_instr  : std_logic_vector(INSTR_WIDTH-1 downto 0);

    -- Decode -> Execute signals
    signal dec_instruction : std_logic_vector(INSTR_WIDTH-1 downto 0);
    signal dec_rs1_data    : std_logic_vector(INSTR_WIDTH-1 downto 0);
    signal dec_rs2_data    : std_logic_vector(INSTR_WIDTH-1 downto 0);
    signal dec_imm32       : std_logic_vector(INSTR_WIDTH-1 downto
0);
    signal dec_rd_addr     : std_logic_vector(4 downto 0);
    signal dec_pc_inc      : std_logic_vector(WIDTH-1 downto 0);

    -- Execute outputs / feedback
    signal exec_branch_en  : std_logic;
    signal exec_alu_result : std_logic_vector(INSTR_WIDTH-1 downto 0);
```

```

    signal exec_rs2_data      :   std_logic_vector(INSTR_WIDTH-1 downto
0);
    signal exec_instr        :   std_logic_vector(INSTR_WIDTH-1 downto
0);
    signal exec_rd_addr      :   std_logic_vector(4 downto 0);
    signal exec_pc_out       :   std_logic_vector(9 downto 0);

    -- Jump address derived from ALU result (for branches/jumps)
    signal jump_addr         :   std_logic_vector(WIDTH-1 downto 0);

    --memory signals
    signal mem_RAM_output    :   std_logic_vector(INSTR_WIDTH-1 downto
0);
    signal mem_reg_ALU      :   std_logic_vector(INSTR_WIDTH-1 downto
0);
    signal mem_instr_out    :   std_logic_vector(INSTR_WIDTH-1 downto
0);
    signal mem_pc_out       :   std_logic_vector(9 downto 0);

    --write back signals
    signal wb_en            :   std_logic;
    signal wb_data          :   std_logic_vector(INSTR_WIDTH-1 downto
0);
    signal wb_addr          :   std_logic_vector(4 downto 0);

begin

    -- Jump address is the lower bits of the ALU result
    -- better to define jump_addr in execute stage
    --jump_addr <= exec_alu_result(WIDTH-1 downto 0);

    -- Fetch Stage
    fetch_inst: fetch
        generic map (
            N => WIDTH,
            M => INSTR_WIDTH
        )
        port map (
            jump_addr    => jump_addr,
            pc_select    => exec_branch_en,

```

```

        rst          => rst,
        clk          => clk,
        decode_addr => internal_pc_inc,
        instruction => internal_instr
    );

-- Decode Stage
decode_inst: entity work.decode
    generic map (
        FUNC_WIDTH => 6,
        ADDR_WIDTH => 5,
        DATA_WIDTH => 32
    )
    port map (
        clk          => clk,
        rst          => rst,
        instruction_in => internal_instr,
        pc_inc       => internal_pc_inc,
        wb_data      => wb_data,
        wb_addr      => wb_addr,
        wb_en        => wb_en,
        rs1_data     => dec_rs1_data,
        rs2_data     => dec_rs2_data,
        sign_ext_imm => dec_imm32,
        rd_addr_out  => dec_rd_addr,
        pc_inc_out   => dec_pc_inc,
        instruction_out => dec_instruction
    );

-- Execute Stage
execute_inst: entity work.execute
    generic map (
        DATA_WIDTH => INSTR_WIDTH,
        PC_WIDTH    => WIDTH
    )
    port map (
        clk          => clk,
        rst          => rst,
        instruction   => dec_instruction,
        pc_inc       => dec_pc_inc,

```

```

        rs1_data      => dec_rs1_data,
        rs2_data      => dec_rs2_data,
        sign_ext_imm  => dec_imm32,
        rd_addr_in    => dec_rd_addr,
        Branch_en     => exec_branch_en,
        ALU_result    => exec_alu_result,
        rs2_data_out   => exec_rs2_data,
        instr_out      => exec_instr,
        rd_addr_out    => exec_rd_addr,
        pc_out         => exec_pc_out,
        jump_addr      => jump_addr
    );

```

-- Memory stage

```

memory_inst:  entity work.memory
    generic map(
        DATA_WIDTH=> INSTR_WIDTH
    )
    port map(
        clk=>clk,
        rst=>rst,
        pc_inc=>exec_pc_out,
        instruction=>exec_instr,
        ALU_result=>exec_alu_result,
        rs2_data=>exec_rs2_data,
        RAM_output=>mem_RAM_output,
        reg_ALU=>mem_reg_ALU,
        instr_out=>mem_instr_out,
        pc_out=>mem_pc_out
    );

```

```

write_back_inst:  entity work.write_back
    generic map(
        DATA_WIDTH=> INSTR_WIDTH
    )
    port map(
        RAM_output=>mem_RAM_output,
        reg_ALU=>mem_reg_ALU,
        pc_inc=>mem_pc_out,
        instruction=>mem_instr_out,

```



```

        wb_en=>wb_en,
        wb_data=>wb_data,
        wb_addr=>wb_addr
    );

end architecture structural;

```

Memory.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
--use work.register_pkg.all;
use work.decode_reg_pkg.all;

entity Memory is
    generic(
        DATA_WIDTH : integer := 32
    );
    port(
        clk : in std_logic;
        rst : in std_logic;

        instruction : in std_logic_vector(DATA_WIDTH-1 downto 0);
        ALU_result : in std_logic_vector(DATA_WIDTH-1 downto 0);
        rs2_data : in std_logic_vector(DATA_WIDTH-1 downto
0);
        pc_inc : in std_logic_vector(9 downto 0);

        RAM_output : out std_logic_vector(DATA_WIDTH-1 downto 0);
        reg_ALU : out std_logic_vector(DATA_WIDTH-1 downto 0);
        instr_out : out std_logic_vector(DATA_WIDTH-1 downto 0);
        pc_out : out std_logic_vector(9 downto 0)
    );
end entity;

```

architecture structural of Memory is

```
    signal trunc_addr    :    std_logic_vector(9 downto 0);
    signal wren           :    std_logic := '0';
begin

    trunc_addr<=ALU_result(9 downto 0);
    wren <= '1' when instruction(31 downto 26) = OP_SW else '0';

    DATA_MEM      :    entity work.factorial_ram
        port map(
            address=>trunc_addr,
            clock=>clk,
            data=>rs2_data,
            wren=>wren,
            q=>RAM_output
        );

    PC_out_reg  :    entity work.reggi
        generic map(
            N=>10
        )
        port map(
            data_in=>pc_inc,
            rst=>rst,
            clk=>clk,
            data_out=>pc_out
        );

    reg_ALU_reg :    entity work.reggi
        generic map(
            N=>32
        )
        port map(
            data_in=> ALU_result,
            rst=>rst,
            clk=>clk,
            data_out=>reg_ALU
        );
```

```

instr_addr_reg : entity work.reggi
  generic map(
    N=>32
  )
  port map(
    data_in=>instruction,
    rst=>rst,
    clk=>clk,
    data_out=>instr_out
  );
end architecture structural;

```

write_back.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
--use work.MUX.pkg.all;
use work.decode_reg_pkg.all;

entity write_back is
  generic(
    DATA_WIDTH : integer := 32
  );
  port(
    RAM_output : in std_logic_vector(DATA_WIDTH-1 downto 0);
    reg_ALU : in std_logic_vector(DATA_WIDTH-1 downto 0);
    instruction : in std_logic_vector(DATA_WIDTH-1 downto 0);
    pc_inc : in std_logic_vector(9 downto 0);

    wb_en : out std_logic;
    wb_data : out std_logic_vector(DATA_WIDTH-1 downto 0);
    wb_addr : out std_logic_vector(4 downto 0)
  );
end entity write_back;

```

architecture structural of write_back is

```
    signal mux_sel    :    std_logic := '0';
    signal opcode     :    std_logic_vector(5 downto 0);
    signal mux_out    :    std_logic_vector(DATA_WIDTH-1 downto 0);
    signal pc_data    :    std_logic_vector(DATA_WIDTH-1 downto 0) := (others
=> '0');
```

begin

```
    pc_data(9 downto 0) <= pc_inc;
    opcode <= instruction(31 downto 26);
    -- address is (25 downto 21) when opcode is not JAL or JALR
    wb_addr <= instruction(25 downto 21) when opcode /= OP_JAL and
                                                    opcode /=
OP_JALR else
                                                    --else hard set to
R31
```

```
                                                    "11111";

    mux_sel <= '1' when opcode = "000001" else '0';
    wb_en    <= '0' when (opcode = OP_NOP or opcode = OP_SW or
                        opcode = OP_BEQZ or opcode = OP_BNEZ or
                        opcode = OP_J    or opcode = OP_JR)
                else '1';
```

```
    --when opcode is not JALR or JAL use rs2 data or alu data
    wb_data <= mux_out when opcode /= OP_JALR and
                                                    opcode /= OP_JAL
                --else use the program counter as data
                else pc_data;
```

```
MUXXY      :    entity work.MUX
    generic map(
        N => 32
    )
    port map(
        A=> RAM_output,
        B=> reg_ALU,
        S=> mux_sel,
```

```

        OUTPUT=> mux_out
    );

end architecture structural;

```

tb_fetch_decode_execute_memWb.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;

entity tb_fetch_decode_execute_memWb is
end entity tb_fetch_decode_execute_memWb;

architecture behavioral of tb_fetch_decode_execute_memWb is

    component DLX_Processor
        generic (
            WIDTH          : integer := 10;
            INSTR_WIDTH    : integer := 32
        );
        port (
            clk             : in  std_logic;
            rst             : in  std_logic
        );
    end component DLX_Processor;

    signal tb_rst : std_logic := '1';

    -- Clock
    signal clk : std_logic := '0';
    constant CLK_PERIOD : time := 10 ns;

begin

    -- Instantiate the DLX Processor (Fetch + Decode + Execute)
    uut : entity work.DLX_Processor

```

```

    generic map (
        WIDTH          => 10,
        INSTR_WIDTH    => 32
    )
    port map (
        clk             => clk,
        rst             => tb_rst
    );

-- Clock process
clk_process : process
begin
    clk <= '0';
    wait for CLK_PERIOD / 2;
    clk <= '1';
    wait for CLK_PERIOD / 2;
end process;

-- =====
-- FACTORIAL PROGRAM IN ROM (factorial_NOops.mif):
-- NOP-padded to avoid data hazards (2 NOPs after every
-- register-writing instruction and every branch/jump).
--
-- 000: 04A00001  LW   R5, n(R0)                [writes R5]
-- 001: 00000000  NOP
-- 002: 00000000  NOP
-- 003: 10800001  ADDI R4, R0, 1                [writes R4]
-- 004: 00000000  NOP
-- 005: 00000000  NOP
-- 006: 081F0000  SW   R4, f(R0)
-- 007: 10E5FFFF  ADDI R7, R5, -1              [writes R7]
-- 008: 00000000  NOP
-- 009: 00000000  NOP
-- 00A: ACE00000  BEQZ R7, done(02C)           [branch]
-- 00B: 00000000  NOP (delay slot)
-- 00C: 00000000  NOP (delay slot)
-- 00D: BC000000  JAL  factorial(016)          [jump, writes R31]
-- 00E: 00000000  NOP (delay slot)
-- 00F: 00000000  NOP (delay slot)
-- 010: 20A50001  SUBI R5, R5, 1              [writes R5]

```

```

-- 011: 00000000 NOP
-- 012: 00000000 NOP
-- 013: B4000000 J    loop(007)                [jump]
-- 014: 00000000 NOP    (delay slot)
-- 015: 00000000 NOP    (delay slot)
-- 016: 04C00000 LW     R6, f(R0)              [writes R6]
-- 017: 00000000 NOP
-- 018: 00000000 NOP
-- 019: 0D002800 ADD     R8, R0, R5             [writes R8]
-- 01A: 00000000 NOP
-- 01B: 00000000 NOP
-- 01C: 10600000 ADDI    R3, R0, 0              [writes R3]
-- 01D: 00000000 NOP
-- 01E: 00000000 NOP
-- 01F: 0C633000 ADD     R3, R3, R6             [writes R3]
-- 020: 00000000 NOP
-- 021: 00000000 NOP
-- 022: 21080001 SUBI    R8, R8, 1              [writes R8]
-- 023: 00000000 NOP
-- 024: 00000000 NOP
-- 025: B1000000 BNEZ    R8, multiply_loop(01F) [branch]
-- 026: 00000000 NOP    (delay slot)
-- 027: 00000000 NOP    (delay slot)
-- 028: 081F0000 SW      R3, f(R0)
-- 029: B800001F JR      R31                    [jump]
-- 02A: 00000000 NOP    (delay slot)
-- 02B: 00000000 NOP    (delay slot)
-- 02C: B4000000 J      done(02C)                [jump]
-- 02D: 00000000 NOP    (delay slot)
-- 02E: 00000000 NOP    (delay slot)
--
-- NOTES:
-- 1. Branches/jumps are SELF-DRIVEN by execute. No testbench
--    control of PC needed. Only writeback is TB-driven.
--
-- 2. Jump target addresses in the MIF are encoded as 0, so all
--    branches/jumps redirect PC to 000 (program restart).
--
-- 3. Without a memory or writeback stage, register values only
--    change when the TB writes them. The NOPs prevent timing

```

```

--      misalignment in the pipeline, but the actual register
--      contents depend entirely on TB writeback signals.
--
-- 4. Pipeline timing (3-stage, approx):
--      Instruction at addr N appears at exec_instr_out ~N+2 cycles
--      after pipeline start. Branch_en effect reaches fetch 1 cycle
--      after it appears at execute output.
--
-- 5. Expected linear flow (R7 nonzero, BEQZ not taken):
--      000-006: setup (LW, NOPs, ADDI, NOPs, SW)
--      007-009: ADDI R7, NOPs
--      00A-00C: BEQZ (not taken), NOPs
--      00D:      JAL → branch_en='1' → PC redirects to 000
--      00E-00F: NOPs (garbage in pipeline, harmless)
--      Then repeats from 000.
-- =====

```

```
stm_process : process
```

```
begin
```

```

-- =====
-- PHASE 1: RESET
-- =====

```

```

-- =====
-- PHASE 2: PRE-POPULATE REGISTERS
-- Write register values immediately after reset.
-- Pipeline is starting but the first instructions that
-- read these registers don't reach decode for several
-- cycles (NOPs give us plenty of margin).
--

```

```

-- Without a writeback stage, the ALU computes results
-- but can't write them to registers. The TB must provide
-- all register values that instructions depend on.
-- =====

```

```

-- R5 = 5 (n, the factorial input)
-- Used by: ADDI R7,R5,-1 at addr 007

```

```
tb_rst <= '1';
```

```
wait for CLK_PERIOD*2;
```



```

tb_rst <= '0';

wait for CLK_PERIOD;

-- R4 = 1 (initial f value)
-- Used by: SW R4,f(R0) at addr 006

wait for CLK_PERIOD;

-- R7 = 4 (n-1, nonzero so BEQZ at 00A is NOT taken)
-- Used by: BEQZ R7 at addr 00A

wait for CLK_PERIOD;

-- R31 = 16 (return address, simulates JAL writeback)
-- Used by: JR R31 at addr 029 (unreachable in first pass)

wait for CLK_PERIOD;

-- R6 = 1 (f loaded from memory)
-- Used by: ADD R3,R3,R6 at addr 01F (unreachable in first pass)

wait for CLK_PERIOD;

-- R8 = 5 (multiply loop counter = n)
-- Used by: SUBI R8,R8,1 at addr 022 (unreachable in first pass)

wait for CLK_PERIOD;

-- R3 = 0 (accumulator)
-- Used by: ADD R3,R3,R6 at addr 01F (unreachable in first pass)

wait for CLK_PERIOD;

-- =====
-- PHASE 3: FIRST PASS (addresses 000 through 00D)
-- The pipeline runs linearly from 000. With R7=4 (nonzero),
-- BEQZ at 00A is NOT taken. JAL at 00D fires branch_en='1'
-- with target=0, looping back to 000.
--

```

```

-- In waveform, verify:
--   - exec_instr_out shows LW, NOP, NOP, ADDI, NOP, NOP, SW,
--     ADDI, NOP, NOP, BEQZ, NOP, NOP, JAL in sequence
--   - alu_result_out for ADDI R4,R0,1 (addr 003) = 0x00000001
--   - alu_result_out for ADDI R7,R5,-1 (addr 007) = 0x00000004
--   - branch_en_out stays '0' for BEQZ (R7 nonzero)
--   - branch_en_out goes '1' for JAL at addr 00D
--   - After JAL, PC redirects to 000 (2 NOPs flush harmlessly)
--
-- 14 instructions (000-00D) + 2 pipeline fill + 2 delay slots
-- = ~18 cycles minimum. Use 20 for margin.
-- =====
wait for CLK_PERIOD * 20;

-- =====
-- PHASE 4: SECOND PASS (pipeline has restarted at 000)
-- Update registers to simulate progress from first pass.
-- The processor loops back through the same instructions.
-- =====

-- R5 = 4 (decremented n, simulating SUBI R5,R5,1)

wait for CLK_PERIOD;

-- R7 = 3 (new n-1 = 4-1)

wait for CLK_PERIOD;

-- R3 = 5 (accumulator after first multiply: 1 * 5 = 5)

wait for CLK_PERIOD;

-- R6 = 5 (f updated)

wait for CLK_PERIOD;

-- R8 = 4 (new loop counter)

wait for CLK_PERIOD;
-- Let second pass run (same flow: 000-00D-loop to 000)

```

```

wait for CLK_PERIOD * 20;

-- =====
-- PHASE 5: BEQZ TAKEN PATH
-- Set R7=0 so BEQZ at 00A fires.
-- This tests that the branch checker correctly evaluates
-- BEQZ when rs1=0 and produces branch_en='1'.
-- Target is still 0, so PC goes to 000 either way,
-- but branch_en will go HIGH at the BEQZ instruction
-- instead of waiting for JAL at 00D.
--
-- Verify in waveform:
--   - branch_en_out goes '1' when BEQZ (opcode 0x2B)
--     appears at exec_instr_out
-- =====

wait for CLK_PERIOD;
-- Let pipeline run: 000→00A(BEQZ taken)→000
-- BEQZ at 00A, faster loop: ~12 instructions + pipeline
wait for CLK_PERIOD * 20;

-- =====
-- PHASE 6: FINAL OBSERVATION
-- Pipeline continues looping. All branches/jumps go to 0.
-- Let it run a few more cycles for waveform capture.
-- =====

wait for CLK_PERIOD * 10;

wait;
end process;

end architecture behavioral;

```